MCP Integration Studio

Technical Documentation | Review edition 0.1 | 28 October 2026

NEW DRAFTS + recovered source evidence. Not an approved implementation baseline.

Contents: 1 Architecture/HLD; 2 LLD; 3 Database ERD; 4 OpenAPI guide, operation and schema reference; 5 Provenance and review; 6 ERD source; 7 Validation.

The ZIP includes editable Markdown, Mermaid and YAML, the unchanged original PRD, and a hash manifest. Original OpenAPI and ERD files were not recovered. See provenance for limits and conflicts.

Architecture / High-Level Design

Status: NEW DRAFT. Retained principles are identified below; component responsibilities and topology are proposals.

1. Purpose and scope

MCP Integration Studio inspects an explicitly selected source revision, builds an evidence-backed architecture view, recommends integration capabilities, obtains developer approval, generates isolated adapters and validates downloadable artifacts. It preserves an application-agnostic core with versioned analyzer/generator plugins. Support for any particular language is a separately tested plugin capability, not a promise to understand every application.

Five retained screens: Dashboard, Project Setup, Architecture Explorer, MCP Studio, Results & Validation. Stitch is the visual reference; OpenAPI is intended to be the HTTP contract reference. The backend must enforce permissions and approval gates.

2. Evidence and baseline boundaries

The recovered conversation describes Next.js/React/TypeScript and a retained Go API, Python workers, PostgreSQL, Redis, Temporal and S3-compatible storage. The recovered PRD instead proposes a CLI-first Node/TypeScript engine with JSON/SQLite persistence. The following Studio topology follows the later conversation as a DRAFT interpretation, pending reconciliation; it does not assert the stack was independently approved. No runtime manifests or implementation were recovered.

3. Logical components

- Web UI: five-screen workflow, evidence navigation, plan review, validation results. Generated API client is a proposal; browser state never authorizes generation.

- Go control API: authenticated access, project ownership, revision checks, plan approval, job initiation, artifact download gates and audit events.

- Analysis workers: read-only acquisition, deterministic parsing, capability discovery, evidence extraction and normalized graph construction.

- Planner and policy evaluator: produce a versioned recommendation from normalized findings; explicit unknowns and blocked capabilities remain visible.

- Generator: turns an approved immutable plan into a separate artifact tree; preserves source application authorization and domain-service boundaries.

- Validation runner: executes allowed checks in an isolated environment; records actual execution evidence and mandatory-check disposition.

- PostgreSQL: proposed source of truth for projects, runs, graph, plans, approvals, artifacts, validation and audits.

- Temporal: proposed durable orchestration; workflow retries must not repeat irreversible effects. Database records expose business status, not workflow history internals.

- Object storage: immutable source snapshots, reports, generated bundles and logs. PostgreSQL stores identifiers and hashes, not binaries or credentials.

- Redis: optional ephemeral cache/progress acceleration; never sole authority for approvals or validation gates. MVP necessity remains unresolved.

4. Data and control flow

1. User registers a local workspace or repository URL and immutable revision. Credentials are resolved by reference outside persisted source metadata.

2. API validates ownership and acquisition policy; a worker creates a read-only snapshot and records its digest.

3. Analyzer plugins publish graph nodes, edges, capabilities and evidence. Unsupported constructs are marked unknown.

4. Planner produces an immutable plan revision, integration type, selected capabilities, configuration digest, policy version and risk rationale.

5. User reviews the exact plan and confirms security. API records an approval bound to plan content hash and actor.

6. Generation atomically checks this approval and schedules work. Generation writes only to isolated output.

7. Validation records passed, failed, skipped or not_run checks. Only all mandatory checks passed permits a validated bundle download.

8. User reviews output. Download does not merge, deploy, push or modify the original application.

5. Trust and security boundaries

Source files, comments, repository instructions, model responses and external server metadata are untrusted data. Analysis must not execute repository scripts. Plugins receive a bounded source view, deadline, memory budget and output schema. Optional model inference requires explicit consent and a documented, redacted payload; deterministic local analysis must remain usable without remote inference.

Authentication mechanism is proposed as bearer access tokens; issuer, audience, identity lifecycle and local bootstrap are unresolved. Every resource lookup is scoped to authenticated tenant and project membership. Tenant IDs are derived by the server, never accepted as authority from request bodies. Composite database keys prevent cross-tenant references; application authorization remains mandatory even if row-level security is later added.

Workers receive no production credentials. Secret references resolve at execution through an approved local secret mechanism; raw tokens must not enter logs, plans or bundles. DB adapters must reuse domain authorization or be blocked; direct database access is not a default escape hatch.

6. Local deployment proposal

Local-first is retained: no paid hosting or cloud deployment required for MVP. A proposed Compose environment runs web, API, isolated workers, PostgreSQL, Temporal and local S3-compatible storage; Redis is optional pending need. Bind control ports to loopback by default. Service data volumes have explicit retention and backup procedures. Do not expose a developer stack to an external network by default.

The exact container images, versions, ports, worker sandbox technology and Windows filesystem strategy remain open. A deployment manifest is not included because none was recovered or validated. Future cloud packaging must preserve the same approval, data and egress boundaries.

7. Reliability and observability

Use request IDs, run IDs, structured redacted logs and append-only audit records. Reconcile stranded queued/running jobs with workflow status after restart. Persist output digests before marking a job successful. Retries are bounded and classified by transient versus deterministic failure; timeouts never imply pass. Use an outbox or equivalent durable enqueue mechanism to avoid a committed run without scheduled work (new proposal).

Performance targets, concurrency caps, retention periods, backup RPO/RTO and availability SLOs require measured baselines. The PRD's under-five-minute report target and fixture correctness target remain unverified proposals.

8. Architecture acceptance

Demonstrate the complete five-screen flow against one immutable revision; reject cross-tenant reads; prove source scripts do not run during scanning; deny generation with stale approval; deny validated download after failed/skipped mandatory checks; verify restart recovery and local operation without remote inference. These are planned acceptance checks, not executed tests.

Low-Level Design

Status: NEW DRAFT. Interfaces, states, constraints and failure behavior below require review.

1. Module contracts

Acquirer.resolve(source, revision, access_reference) returns Snapshot(id, digest, immutable_root). It canonicalizes local roots, rejects traversal/symlink escapes and applies ignore/size limits. Acquisition of remote repositories is distinct from analysis and subject to egress policy.

Analyzer.analyze(snapshot, plugin_version, budget) returns graph, candidates, evidence and diagnostics. Each evidence reference carries source path, source digest, line interval and detector version. Missing evidence cannot be promoted to certainty. Plugin failure produces a partial run with explicit diagnostics; it never invents capabilities.

Planner.build(analysis_run, integration_type, capability_ids, config, policy_version) returns immutable Plan(version, content_hash, rationale, risks). Recommended type and developer-selected type are separate data. A new plan version is created for every material change.

Generator.generate(plan, approval, snapshot) returns immutable artifact metadata. It receives only verified references and permitted configuration. Candidate tools must retain input/output schema, authorization context and domain-service mapping. A dangerous or unsupported mapping blocks generation.

Validator.validate(generation_run, policy_version) returns named CheckResult records with status, mandatory flag, command/tool version, exit code, start/end times and log reference. Compile, schema, authorization and protocol checks are policy-driven; skipped or not_run is never passed.

2. Business states

Analysis: queued -> running -> succeeded | partial | failed | cancelled. Only succeeded analysis may create a plan in this draft; partial output remains inspectable. Allowing reviewed partial analysis is an explicit future decision.

Plan: draft -> approved -> superseded. Editing creates a new version; old approvals never carry forward. Revocation and regeneration policy is open; this draft treats superseded plans as ineligible for new work.

Generation: queued -> running -> succeeded | failed | cancelled. It produces artifacts but does not imply validation success.

Validation: queued -> running -> succeeded | failed | cancelled. Individual checks: passed | failed | skipped | not_run. Validation succeeds only if every mandatory check exists and passed and the runner finished normally. Required-check list must come from a pinned policy, preventing success from an empty check list.

These are proposed internal/API states. They do not claim the conflicting Stitch labels or PRD's combined state sequence are already reconciled.

3. Approval and generation transaction

BEGIN; load tenant-scoped plan FOR UPDATE; require approved status, current plan version, matching source/config/policy hashes and exact approval hash; authorize current actor; insert generation run and outbox event; append audit event; COMMIT. The outbox dispatcher starts one durable workflow keyed by generation ID. Unique constraints prevent duplicate dispatch effects. Network retries after uncertain commits require client readback; request-idempotency storage is not specified in this draft.

Plan hash is SHA256 over a canonical serialization of source digest, analysis/plugin versions, selected integration type, capabilities, schemas, config with secret references, policy version and generator version. Exact canonicalization format must be pinned before interoperability testing. The confirmation UI sends the server-issued hash and plan version; the server recomputes/compares it.

Changing integration type must invalidate confirmation per recovered requirements. This draft proposes also invalidating on source, capability, adapter, transport, configuration, policy or generator changes. The expanded set is NEW and needs ratification.

4. Persistence and boundaries

Store each business row under tenant_id; use composite foreign keys (tenant_id, resource_id). Resource IDs are UUIDs. Each project has many analysis runs; every plan belongs to one analysis run; every generation references one plan and one approval; artifacts belong to a generation; validations belong to a generation and a pinned policy. See database narrative for cardinalities and enforcement gaps.

Object writes use temporary keys followed by digest verification and publication of immutable metadata. A failed upload cannot mark a run succeeded. Downloads proxy through the API after tenant authorization and validation checks; do not return permanent public object URLs.

5. API implementation behavior

Authenticate before resource existence disclosure. Return 401 for missing/invalid authentication, 403 for known forbidden operations, 404 for absent or inaccessible resources according to a consistent anti-enumeration policy, 409 for stale plan/approval/state conflicts, 422 for semantically invalid requests and 429 for limits. Error bodies contain code, message and request_id without secrets or raw stack traces.

List operations use opaque cursor pagination with a bounded limit. Only project lists and artifact lists are exposed in this draft; a dashboard summary endpoint, graph query pagination, cancellation and event streaming remain extensions. The current graph response is bounded by analysis policy; large-graph slicing is unresolved.

6. Isolation and filesystem controls

Read source snapshots only. Generate in a separate output root; normalize every emitted path, reject absolute paths, traversal, symlinks and collisions. Validation runs with explicit CPU/memory/time limits, a temporary workspace, restricted network and approved commands. Source writes, dependency installs, production calls and package publication are separate actions requiring defined policy and consent.

7. Failure recovery

Worker crash: orchestration retries bounded deterministic activities with stable run IDs. Policy/schema failure: preserve diagnostics, mark failed, require user correction. Storage failure: retain pending status and retry publication. Cancellation is modeled but no cancellation endpoint is included yet. A later implementation must specify cooperative cancellation and cleanup before exposing it.

8. Planned test matrix

- Unit: canonical plan hashes; path handling; schema mapping; mandatory-check gate with empty, skipped and failed checks.

- Integration: tenant-scoped composite references; approval/generation race; object metadata publication; duplicate dispatch; workflow restart.

- End-to-end: create project, analyze fixture, inspect evidence, create/revise/approve plan, generate, validate and download.

- Security: cross-tenant resource IDs, stale approval replay, source prompt injection, malicious repository script, secret leakage and unauthorized domain invocation.

- Compatibility: pin MCP version/SDK/transport and exercise generated artifacts against a selected client; no compatibility claim is made here.

Database ERD and persistence design

Status: NEW DRAFT PostgreSQL logical model; not recovered schema or executable migration.

Entities and relationships

Tenant owns Projects and Memberships; User has Memberships in Tenants. Project has AnalysisRuns; AnalysisRun has GraphNodes, GraphEdges and Plans. Plan has Approvals and GenerationRuns. Approval authorizes GenerationRuns. GenerationRun has Artifacts and ValidationRuns. ValidationRun has ValidationChecks. Tenant has AuditEvents.

Each child belongs to exactly one parent. A parent may have zero or many children. Generation references exactly one plan and one approval; approval must refer to that same plan. Graph edges connect two nodes in the same analysis run. Tenant ownership is inherited and stored explicitly for enforcement.

Key and constraint policy

All entities except membership have UUID primary keys; membership uses (tenant_id,user_id). Tenant-owned rows additionally expose UNIQUE(tenant_id,id). Every parent reference uses (tenant_id,parent_id) foreign keys. Graph edges require composite references (tenant_id,analysis_run_id,from_node_id/to_node_id) to the corresponding node keys. Generation's (tenant_id,plan_id,approval_id) must reference an approval key covering those columns, preventing approval from another plan. These constraints are narrative requirements; the Mermaid file illustrates the logical model, not complete DDL.

Plan UNIQUE(tenant_id,analysis_run_id,version). Version is a positive integer; content_hash is a 64-character SHA256 hex string. Approval stores actor, plan hash and timestamp. Authorization must compare approval hash with immutable plan content; a foreign key alone cannot enforce it. A user may be removed from membership while historic approvals remain as audit evidence; current rights are checked again before generation.

Run/artifact timestamps are UTC. ValidationCheck UNIQUE(tenant_id,validation_run_id,name); mandatory-check names come from pinned policy, not user input. Terminal state transitions require completed_at. Artifact paths are relative; content digest, media type, byte count and storage key are required. Graph/evidence JSON is versioned and schema-validated before persistence.

Indexes and transaction choices

Proposed indexes: projects(tenant_id,created_at,id), analysis_runs(tenant_id,project_id,created_at), plans(tenant_id,analysis_run_id,version), generation_runs(tenant_id,plan_id), artifacts(tenant_id,generation_run_id), validations(tenant_id,generation_run_id,created_at), audit_events(tenant_id,created_at). Index graph lookups by analysis run and endpoints. Benchmark JSON indexing before adding broad GIN indexes.

Use optimistic version checks for plan approval and row locking for generation admission. Outbox delivery needs an additional operational table or workflow-equivalent durable mechanism; it is deliberately not represented as a product entity in this ERD. No migration or performance verification is supplied.

Data lifecycle

No cascade deletion of approvals, validations or audits is proposed. Project deletion is not exposed by the draft API. Retention periods, tenant erasure, user anonymization, source snapshot lifetime, artifact expiry, encrypted backup and restore procedures need explicit decisions. Avoid retaining source contents in graph JSON; store source evidence references and optional minimized snippets subject to policy. Secrets never belong in these tables.

Model/API mapping

Project -> Project; AnalysisRun plus graph -> Analysis/Graph; Plan -> Plan; Approval -> Approval; GenerationRun -> Generation; Artifact -> Artifact; ValidationRun and ValidationCheck -> Validation/Check. Tenant/User/Membership/AuditEvent are internal entities; identity/admin endpoints are outside this draft. Source/config/policy hashes tie these records to the same immutable input.

OpenAPI specification guide

Status: NEW DRAFT. No recovered Studio HTTP API was available. specs/openapi.yaml is a proposed OpenAPI 3.0.3 contract and must be reviewed before client generation or implementation.

Protocol boundary and rules

This is the Studio's HTTP control plane, not an MCP protocol specification. MCP SDK/version/transports remain unresolved. The example local base URL is a proposal, not a running service. All operations require bearer authentication and server-derived tenant context. Raw credentials and tenant IDs are excluded from write bodies.

Project source revision must resolve immutably; local inputs require a snapshot digest. credential_reference refers to a separately provisioned secret, never its value. Graph nodes represent discoverable capabilities where applicable; capability_ids must identify eligible nodes in the plan's analysis. Plugin configuration is validated against the selected plugin schema; the generic JSON boundary is not a security validation substitute.

Create plans instead of editing immutable versions. Current draft assigns monotonically increasing versions per analysis and supersedes older plans transactionally; approving or generating a superseded plan returns 409. All material edits require a fresh plan and approval. Integration type none produces an inspectable recommendation but cannot generate adapters.

Async operations return 202 and a resource ID to poll. A failed/partial analysis is not success; only succeeded analysis can create plans. A generation must succeed before validation. Validation policy is selected by the server from the approved plan. Download checks the exact generation and policy; a superseded validation for different artifacts cannot unlock it.

Known contract limitations

No token lifecycle, user/tenant administration, secret provisioning, retry idempotency, cancellation API, progress events, artifact preview, draft-download endpoint, graph slicing or project deletion is specified. None is implied by a UI placeholder. Error coverage is generic; exact per-operation error codes and examples remain review work. Live server behavior and MCP compatibility have not been tested.

Operation reference

- GET /projects: listProjects; request none; response 200 ProjectList.
- POST /projects: createProject; request ProjectInput; response 201 Project.
- GET /projects/{project_id}: getProject; request none; response 200 Project.
- POST /analyses: startAnalysis; request AnalysisInput; response 202 Analysis.
- GET /analyses/{analysis_id}: getAnalysis; request none; response 200 Analysis.
- GET /analyses/{analysis_id}/graph: getGraph; request none; response 200 Graph.
- POST /plans: createPlan; request PlanInput; response 201 Plan.
- GET /plans/{plan_id}: getPlan; request none; response 200 Plan.
- POST /plans/{plan_id}/approvals: approvePlan; request ApprovalInput; response 201 Approval.
- POST /generations: startGeneration; request GenerationInput; response 202 Generation.
- GET /generations/{generation_id}: getGeneration; request none; response 200 Generation.
- POST /validations: startValidation; request ValidationInput; response 202 Validation.
- GET /validations/{validation_id}: getValidation; request none; response 200 Validation.
- GET /artifacts: listArtifacts; request none; response 200 ArtifactList.
- GET /generations/{generation_id}/bundle: downloadBundle; request none; response 200 binary ZIP.

Schema reference

Error

{

"type": "object",

"additionalProperties": false,

"properties": {

"code": {

"type": "string"

},

"message": {

"type": "string"

},

"request_id": {

"type": "string"

}

},

"required": [

"code",

"message",

"request_id"

]

}

Source

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "kind": {
      "type": "string",
      "enum": [
        "local",
        "git"
      ]
    },
    "location": {
      "type": "string"
    },
    "revision": {
      "type": "string"
    },
    "credential_reference": {
      "type": "string"
    }
  },
  "required": [
    "kind",
    "location",
    "revision"
  ]
}
```

ProjectInput

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "name": {
      "type": "string",
```

```
        "minLength": 1,
        "maxLength": 200
      },
      "source": {
        "$ref": "#/components/schemas/Source"
      }
    },
    "required": [
      "name",
      "source"
    ]
  }
```

Project

```
  {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "id": {
        "type": "string",
        "format": "uuid"
      },
      "name": {
        "type": "string"
      },
      "source": {
        "$ref": "#/components/schemas/Source"
      },
      "created_at": {
        "type": "string",
        "format": "date-time"
      }
    },
    "required": [
      "id",
      "name",
      "source",
      "created_at"
    ]
  }
```

ProjectList

```
  {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "items": {
        "type": "array",
        "items": {
          "$ref": "#/components/schemas/Project"
        }
      },
      "next_cursor": {
        "type": "string"
      }
    },
    "required": [
      "items"
    ]
  }
```

AnalysisInput

```
  {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "project_id": {
        "type": "string",
        "format": "uuid"
      }
    },
    "required": [
      "project_id"
    ]
  }
```

Analysis

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "id": {
      "type": "string",
      "format": "uuid"
    },
    "project_id": {
      "type": "string",
      "format": "uuid"
    },
    "status": {
      "type": "string",
      "enum": [
        "queued",
        "running",
        "succeeded",
        "partial",
        "failed",
        "cancelled"
      ]
    },
    "source_digest": {
      "type": "string",
      "pattern": "^[a-f0-9]{64}$"
    },
    "diagnostics": {
      "type": "array",
      "items": {
        "type": "string"
      }
    }
  },
  "required": [
    "id",
    "project_id",
    "status"
  ]
}
```

Evidence

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "path": {
      "type": "string"
    },
    "source_digest": {
      "type": "string",
      "pattern": "^[a-f0-9]{64}$"
    },
    "line_start": {
      "type": "integer",
      "minimum": 1
    },
    "line_end": {
      "type": "integer",
      "minimum": 1
    },
    "detector_version": {
      "type": "string"
    }
  },
  "required": [
    "path",
    "source_digest",
    "detector_version"
  ]
}
```

Node

```
{
  "type": "object",
  "additionalProperties": false,
```

```
  "properties": {
    "id": {
      "type": "string",
      "format": "uuid"
    },
    "kind": {
      "type": "string"
    },
    "label": {
      "type": "string"
    },
    "evidence": {
      "type": "array",
      "items": {
        "$ref": "#/components/schemas/Evidence"
      }
    }
  },
  "required": [
    "id",
    "kind",
    "label",
    "evidence"
  ]
}
```

Edge

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "id": {
      "type": "string",
      "format": "uuid"
    },
    "from_node_id": {
      "type": "string",
      "format": "uuid"
    },
    "to_node_id": {
      "type": "string",
      "format": "uuid"
    },
    "kind": {
      "type": "string"
    }
  },
  "required": [
    "id",
    "from_node_id",
    "to_node_id",
    "kind"
  ]
}
```

Graph

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "analysis_id": {
      "type": "string",
      "format": "uuid"
    },
    "nodes": {
      "type": "array",
      "items": {
        "$ref": "#/components/schemas/Node"
      }
    },
    "edges": {
      "type": "array",
      "items": {
        "$ref": "#/components/schemas/Edge"
      }
    }
  },
  "required": [
    "analysis_id",
```

```
    "nodes",
    "edges"
  ]
}
```

Configuration

```
{
  "type": "object",
  "description": "Plugin-versioned non-secret configuration; resolve only secret references.
Plugin schema validation is mandatory.",
  "additionalProperties": true
}
```

PlanInput

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "analysis_id": {
      "type": "string",
      "format": "uuid"
    },
    "integration_type": {
      "type": "string",
      "enum": [
        "server",
        "client",
        "both",
        "none"
      ]
    },
    "capability_ids": {
      "type": "array",
      "items": {
        "type": "string",
        "format": "uuid"
      }
    },
    "configuration": {
      "$ref": "#/components/schemas/Configuration"
    }
  },
  "required": [
    "analysis_id",
    "integration_type",
    "capability_ids",
    "configuration"
  ]
}
```

Plan

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "analysis_id": {
      "type": "string",
      "format": "uuid"
    },
    "integration_type": {
      "type": "string",
      "enum": [
        "server",
        "client",
        "both",
        "none"
      ]
    },
    "capability_ids": {
      "type": "array",
      "items": {
        "type": "string",
        "format": "uuid"
      }
    },
    "configuration": {
      "$ref": "#/components/schemas/Configuration"
```

```
    },
    "id": {
      "type": "string",
      "format": "uuid"
    },
    "version": {
      "type": "integer",
      "minimum": 1
    },
    "status": {
      "type": "string",
      "enum": [
        "draft",
        "approved",
        "superseded"
      ]
    },
    "content_hash": {
      "type": "string",
      "pattern": "^[a-f0-9]{64}$"
    },
    "policy_version": {
      "type": "string"
    },
    "rationale": {
      "type": "string"
    },
    "risks": {
      "type": "array",
      "items": {
        "type": "string"
      }
    }
  },
  "required": [
    "analysis_id",
    "integration_type",
    "capability_ids",
    "configuration",
    "id",
    "version",
    "status",
    "content_hash",
    "policy_version",
    "rationale",
    "risks"
  ]
}
```

ApprovalInput

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "plan_version": {
      "type": "integer",
      "minimum": 1
    },
    "plan_hash": {
      "type": "string",
      "pattern": "^[a-f0-9]{64}$"
    },
    "security_confirmed": {
      "type": "boolean",
      "enum": [
        true
      ]
    }
  },
  "required": [
    "plan_version",
    "plan_hash",
    "security_confirmed"
  ]
}
```

Approval

```
{
  "type": "object",
```

```
  "additionalProperties": false,
  "properties": {
    "id": {
      "type": "string",
      "format": "uuid"
    },
    "plan_id": {
      "type": "string",
      "format": "uuid"
    },
    "plan_hash": {
      "type": "string",
      "pattern": "^[a-f0-9]{64}$"
    },
    "actor_id": {
      "type": "string",
      "format": "uuid"
    },
    "approved_at": {
      "type": "string",
      "format": "date-time"
    }
  },
  "required": [
    "id",
    "plan_id",
    "plan_hash",
    "actor_id",
    "approved_at"
  ]
}
```

GenerationInput

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "plan_id": {
      "type": "string",
      "format": "uuid"
    },
    "approval_id": {
      "type": "string",
      "format": "uuid"
    },
    "plan_hash": {
      "type": "string",
      "pattern": "^[a-f0-9]{64}$"
    }
  },
  "required": [
    "plan_id",
    "approval_id",
    "plan_hash"
  ]
}
```

Generation

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "id": {
      "type": "string",
      "format": "uuid"
    },
    "plan_id": {
      "type": "string",
      "format": "uuid"
    },
    "approval_id": {
      "type": "string",
      "format": "uuid"
    },
    "status": {
      "type": "string",
      "enum": [
        "queued",
        "running",
```

```
        "succeeded",
        "failed",
        "cancelled"
      ]
    }
  },
  "required": [
    "id",
    "plan_id",
    "approval_id",
    "status"
  ]
}
```

ValidationInput

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "generation_id": {
      "type": "string",
      "format": "uuid"
    }
  },
  "required": [
    "generation_id"
  ]
}
```

Check

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "name": {
      "type": "string"
    },
    "mandatory": {
      "type": "boolean"
    },
    "status": {
      "type": "string",
      "enum": [
        "passed",
        "failed",
        "skipped",
        "not_run"
      ]
    },
    "command": {
      "type": "string"
    },
    "tool_version": {
      "type": "string"
    },
    "exit_code": {
      "type": "integer"
    },
    "started_at": {
      "type": "string",
      "format": "date-time"
    },
    "completed_at": {
      "type": "string",
      "format": "date-time"
    },
    "log_reference": {
      "type": "string"
    }
  },
  "required": [
    "name",
    "mandatory",
    "status"
  ]
}
```

Validation

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "id": {
      "type": "string",
      "format": "uuid"
    },
    "generation_id": {
      "type": "string",
      "format": "uuid"
    },
    "status": {
      "type": "string",
      "enum": [
        "queued",
        "running",
        "succeeded",
        "failed",
        "cancelled"
      ]
    },
    "policy_version": {
      "type": "string"
    },
    "checks": {
      "type": "array",
      "items": {
        "$ref": "#/components/schemas/Check"
      }
    }
  },
  "required": [
    "id",
    "generation_id",
    "status",
    "policy_version",
    "checks"
  ]
}
```

Artifact

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "id": {
      "type": "string",
      "format": "uuid"
    },
    "generation_id": {
      "type": "string",
      "format": "uuid"
    },
    "path": {
      "type": "string"
    },
    "sha256": {
      "type": "string",
      "pattern": "^[a-f0-9]{64}$"
    },
    "media_type": {
      "type": "string"
    },
    "size_bytes": {
      "type": "integer",
      "minimum": 0
    }
  },
  "required": [
    "id",
    "generation_id",
    "path",
    "sha256",
    "media_type",
    "size_bytes"
  ]
}
```

ArtifactList

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "items": {
      "type": "array",
      "items": {
        "$ref": "#/components/schemas/Artifact"
      }
    },
    "next_cursor": {
      "type": "string"
    }
  },
  "required": [
    "items"
  ]
}
```

Provenance, reconciliation and review register

Recovered evidence

Conversation: MCPStudio, 6adf8000-7004-83ed-b840-4aee7adb5739. Retrieved 28 October 2026: available turns include implementation kickoff, mini-step proposal, documentation requests and proposed frontend stack. Retrieval returned no older cursor or attachments; this does not establish that older files never existed.

Agent message 9095aeb7-8bc7-4be5-9484-d2fedd1beeb6 describes proposed frontend tooling and retained backend stack; ecf760d1-f32b-4ff4-98f4-eedd5eee5eba preserves local-first, five screens and application-agnostic principles; 7e698c70-06bb-4882-ab44-48fe50e8cc2d distinguishes proposed roadmap from implementation.

Recovered from sibling chat output M0.1: PRD original PDF, decisions.md, inventory.md, prd-intake.md and source-conversation.json if available. Copies preserve bytes; manifest records SHA256. The PRD is explicitly a draft. Inventory ADR IDs are local tracking references, not original accepted ADRs. Earlier chat 'Assemble MCPStudio specifications' found Library candidates but did not recover the claimed enterprise scaffold or detailed specifications. Its description of a candidate is not evidence of its contents or approval.

Bounded search result

Filename searches in Documents/Codex and Documents found no original openapi.yaml or database-erd.mmd. Checked C:/Users/ADMIN/mcpStudio: README, Git metadata and PRD only at top level. Relevant Downloads candidates were a demo video and Stitch workspace ZIP, not the backend scaffold. The prior inventory reports connected Drive searches returned no relevant inputs. Library originals and uninspected Git branches/history remain possible sources; this package does not claim exhaustive account-wide recovery.

Authorship register

- Original PRD PDF: existing source, itself draft, unchanged.

- Inventory/decisions/intake/conversation: existing secondary evidence, unchanged copies.

- HLD, LLD, database narrative, Mermaid ERD, OpenAPI YAML/API guide: newly authored in this task, review drafts.

- PDF, validation and manifests: newly generated packaging/evidence, not architecture approval.

Decisions still requiring review

R01: Reconcile CLI-first Node/SQLite PRD against five-screen Studio Go/Python/PostgreSQL narrative. Do not silently replace either as approved.

R02: Ratify concrete stack, local topology, required versus optional Redis/Temporal and Windows sandbox support.

R03: Define initial analyzer support matrix while preserving application-agnostic extension contracts.

R04: Pin MCP protocol, SDK and transport compatibility; old UI examples are not interoperability evidence.

R05: Ratify plan hash canonicalization, invalidation set, revocation and concurrency behavior.

R06: Select identity issuer/bootstrap, tenant membership roles, secrets mechanism and optional inference consent/egress policy.

R07: Approve physical schema/migrations, JSON contracts, data lifecycle and audit retention.

R08: Review all proposed endpoints, asynchronous job semantics, error taxonomy, graph limits and omitted UI capabilities.

R09: Define mandatory validation checks, draft versus validated export rights and sandbox/network/install policy.

R10: Resolve Stitch status/token conflicts; retain five screens. Define measured acceptance targets and evidence required for milestone approval.

Review owner for each item: product/architecture/security as appropriate, not yet assigned. Approval/date: pending. This documentation task does not close M0.1 or authorize implementation milestones.

Database ERD - editable Mermaid source

```
%% NEW DRAFT - not recovered or approved; composite tenant constraints are in docs/03-Database-ERD.md
erDiagram
  TENANT ||--o{ MEMBERSHIP : has
  USER ||--o{ MEMBERSHIP : has
  TENANT ||--o{ PROJECT : owns
  PROJECT ||--o{ ANALYSIS_RUN : analyzes
  ANALYSIS_RUN ||--o{ GRAPH_NODE : contains
  ANALYSIS_RUN ||--o{ GRAPH_EDGE : contains
  GRAPH_NODE ||--o{ GRAPH_EDGE : source
  GRAPH_NODE ||--o{ GRAPH_EDGE : target
  ANALYSIS_RUN ||--o{ PLAN : informs
  PLAN ||--o{ APPROVAL : receives
  USER ||--o{ APPROVAL : records
  PLAN ||--o{ GENERATION_RUN : generates
  APPROVAL ||--o{ GENERATION_RUN : authorizes
  GENERATION_RUN ||--o{ ARTIFACT : produces
  GENERATION_RUN ||--o{ VALIDATION_RUN : validates
  VALIDATION_RUN ||--o{ VALIDATION_CHECK : contains
  TENANT ||--o{ AUDIT_EVENT : records
  USER ||--o{ AUDIT_EVENT : acts
  TENANT {
    uuid id PK
    string name
  }
  USER {
    uuid id PK
    string subject UK
  }
  MEMBERSHIP {
    uuid tenant_id PK,FK
    uuid user_id PK,FK
    string role
  }
  PROJECT {
    uuid id PK
    uuid tenant_id FK
    string name
    json source
    datetime created_at
  }
  ANALYSIS_RUN {
    uuid id PK
    uuid tenant_id FK
    uuid project_id FK
    string status
    string source_digest
    json diagnostics
    datetime created_at
  }
  GRAPH_NODE {
    uuid id PK
    uuid tenant_id FK
    uuid analysis_run_id FK
    string kind
    json evidence
  }
  GRAPH_EDGE {
    uuid id PK
    uuid tenant_id FK
    uuid analysis_run_id FK
    uuid from_node_id FK
    uuid to_node_id FK
    string kind
  }
  PLAN {
    uuid id PK
    uuid tenant_id FK
    uuid analysis_run_id FK
    int version
    string status
    string integration_type
    string content_hash
    json configuration
    string policy_version
  }
  APPROVAL {
    uuid id PK
    uuid tenant_id FK
    uuid plan_id FK
```

```
  uuid actor_id FK
  string plan_hash
  datetime approved_at
}
GENERATION_RUN {
  uuid id PK
  uuid tenant_id FK
  uuid plan_id FK
  uuid approval_id FK
  string status
  datetime created_at
}
ARTIFACT {
  uuid id PK
  uuid tenant_id FK
  uuid generation_run_id FK
  string path
  string storage_key
  string sha256
  string media_type
  int size_bytes
}
VALIDATION_RUN {
  uuid id PK
  uuid tenant_id FK
  uuid generation_run_id FK
  string status
  string policy_version
  datetime created_at
}
VALIDATION_CHECK {
  uuid id PK
  uuid tenant_id FK
  uuid validation_run_id FK
  string name
  boolean mandatory
  string status
  int exit_code
  string log_reference
}
AUDIT_EVENT {
  uuid id PK
  uuid tenant_id FK
  uuid actor_id FK
  string action
  string resource_id
  datetime created_at
}
```

Documentation validation report

28 October 2026. Documentation checks only; no application implementation was available.

PASS: OpenAPI uses JSON-compatible YAML syntax and parses; all internal schema references resolve; operation IDs are unique; every template path parameter is required and declared; all Mermaid relationship entities exist. The API has 15 operations and 23 schemas; ERD has 14 entities. Original PRD copied unchanged (SHA256 recorded in validation-results.json and manifest).

Cross-document review: each public persistence-backed resource maps to a logical entity; graph IDs map to node/edge records; approval and generation both bind to plan hash; validation pins policy and generation; tenant context is server-derived throughout. Proposed API modes include none but reject generation for none; this differs from the prototype and remains explicit review scope. Entity IDs use UUIDs. Error responses share one schema. No missing original has been labeled recovered or approved.

LIMITATIONS: no full OpenAPI meta-schema/semantic validator, API implementation, migrations, database constraints, protocol tests or Mermaid renderer were executed. Mermaid source is editable and a text relationship reference is in the PDF. The PDF is rendered and visually reviewed separately before delivery. Structural consistency does not establish approved requirements, production readiness or complete UI coverage. See API omissions and review items R01-R10.